

Clase 8: Modelos de objetos e invariantes

En este tema se consolidan muchas de las ideas fundamentales sobre objetos, representaciones y abstracción, tratadas en temas anteriores. Explicaremos detalladamente la notación gráfica del modelado de objetos y repasaremos los invariantes de representación, las funciones de abstracción y la exposición de representación. Después de leer este tema, es posible que desee volver a los temas del principio para darles un repaso, ya que éstos incluyen más detalles en relación a los ejemplos tratados aquí.

8.1 Modelos de objetos

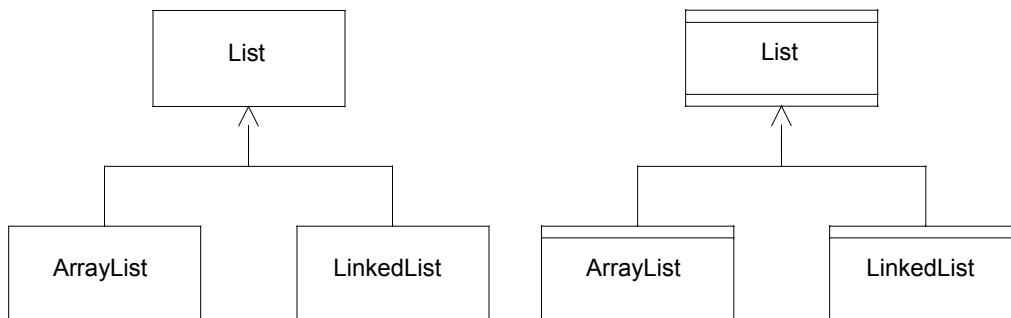
Un *modelo de objeto* es una descripción de una colección de configuraciones. En este tema, nos centraremos en modelos de objeto en forma de código, en los que las configuraciones son estados de un programa. Sin embargo, a lo largo de la asignatura, veremos que se puede utilizar una misma notación, de forma más genérica, para describir cualquier tipo de configuración, como el formato de un sistema de archivos, una jerarquía de seguridad, una topología de red, etc.

Las nociones básicas que yacen bajo los modelos de objeto son increíblemente simples: conjuntos de objetos y las relaciones entre ellos. Lo más complicado para los estudiantes es aprender a construir un modelo útil: cómo capturar las partes interesantes y complicadas de un programa y cómo no entusiasmarse con el modelado de las partes irrelevantes y acabar ante un modelo enorme y de difícil manejo, o por el contrario, decir muy poco, y verse ante un modelo que resulta inútil.

Tanto los modelos de objeto como los diagramas de dependencia de módulos, poseen recuadros y flechas. He aquí la única similitud entre ellos. Bueno, de acuerdo, admito que existen algunas conexiones sutiles entre el modelo de objeto y el diagrama de dependencia de módulos de un programa. Sin embargo, a primera vista, es mejor pensar en ellos como si fuesen completamente diferentes. El diagrama de dependencia de módulos aborda la estructura sintáctica, es decir, las descripciones textuales que existen, y cómo éstas están relacionadas entre sí. El modelo de objeto se centra en la estructura semántica, es decir, qué configuraciones se crean en tiempo de ejecución, y qué propiedades poseen.

8.1.1 Clasificación

Un modelo de objeto expresa dos tipos de propiedades: la clasificación de objetos y las relaciones entre ellos. Para expresar la clasificación, dibujamos un recuadro para cada clase de objetos. En un modelo de objeto en forma de código, estos recuadros corresponderán a las clases e interfaces de Java; en una definición más general, simplemente representarán clasificaciones arbitrarias.



Una flecha con la punta gruesa y cerrada, desde la clase *A* hasta la clase *B*, indica que *A* representa un subconjunto de *B*: es decir, todo *A* es también un *B*. Para demostrar que dos recuadros representan subconjuntos distintos, hacemos que éstos compartan la misma punta de flecha. En el diagrama de arriba, *LinkedList* y *ArrayList* son subconjuntos distintos de *List*.

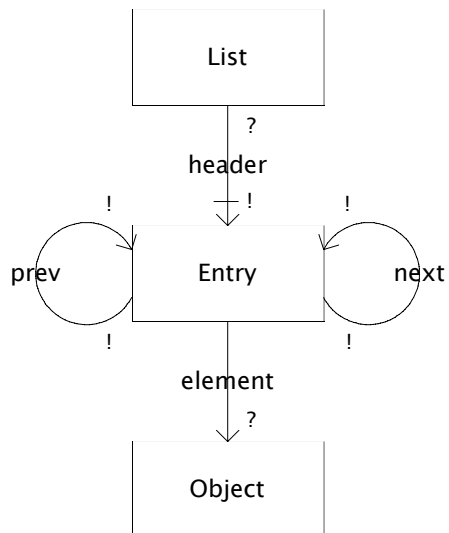
En Java, cada declaración *implements* y *extends* da como resultado una relación de subconjuntos en un modelo de objetos. Ésta es una propiedad del sistema de tipos: si un objeto *o* se crea con un constructor de una clase *C*, y *C* extiende a *D*, entonces se considera que *o* también posee al tipo *D*. El diagrama de arriba muestra el modelo de objetos a la izquierda.

El de la derecha es un diagrama de dependencia de módulos. Sus recuadros representan descripciones textuales, como el código de las clases. Sus flechas, tal como usted recordará, representan la relación "meet" (satisfacer). Por tanto, la flecha que parte de *ArrayList* hacia *List* indica que el código de *ArrayList* satisface la especificación *List*. Dicho de otro modo, los objetos de la clase *ArrayList* se comportan como listas abstractas. Esta es una propiedad sutil que es verdadera debido a los detalles del código. Como veremos más adelante en el tema sobre subtipado (o derivación), es fácil engañarse con esta característica y crear una clase que extiende o implementa a otra sin que exista una relación "meet" entre ellas (en un diagrama de dependencia de módulos, el compartir la punta de la flecha no tiene ninguna importancia).

8.1.2 Campos

Una flecha con una punta abierta desde *A* hacia *B* indica que existe una relación entre los objetos de *A* y los de *B*. Dado que pueden existir muchas relaciones entre dos clases, le damos nombre a las relaciones y etiquetamos las flechas con los nombres. Un campo *f* en una clase *A*, cuyo tipo es *B*, da como resultado una flecha desde *A* hasta *B* etiquetada como *f* (el nombre del campo).

Por ejemplo, el siguiente código produce estructuras que pueden ilustrarse a través del diagrama que se muestra a continuación (ignore por el momento las marcas al final de las flechas):



```

class LinkedList implements List {
    Entry header;
    ...
}
  
```

```

class Entry {
    Entry next;
    Entry prev;
    Object elt;
    ...
}
  
```

8.1.3 Multiplicidad

Hasta ahora, hemos visto la *clasificación* de los objetos en clases y las *relaciones* que muestran que los objetos de una clase pueden estar relacionados con los objetos de otra. Una cuestión básica sobre la relación entre las clases es la *multiplicidad*: cuántos objetos de una clase pueden estar relacionados con un determinado objeto de otra clase.

Los símbolos de multiplicidad son:

- * (cero o más)
- + (uno o más)
- ? (cero o uno)
- ! (exactamente uno).

Cuando se omite un símbolo, * es el símbolo que se asume por defecto (que no indica nada). La interpretación de estas marcas consiste en que cuando hay una marca n en el final B de una flecha de campo f que parte de la clase A hacia la clase B , existen n miembros de la clase B asociados por f con cada A . Esto también funciona al revés; si hay una marca m en el inicio A de una flecha de campo f que parte desde A hacia B , cada B es asociada por los m miembros de la clase A .

En el final de la flecha, es decir, hacia donde mira la punta de la misma, la multiplicidad le indica cuántos objetos puede referenciar una variable. Hasta ahora, no hemos asignado ningún uso a las marcas * y +, pero veremos cómo éstas se utilizan con campos abstractos. La elección de ? o ! depende de si un campo puede o no ser *null*. Al inicio de la flecha, la multiplicidad señala cuántos objetos pueden apuntar a un determinado objeto. Dicho de otro modo, nos da información sobre el hecho de compartir.

Observemos algunas de las flechas y veamos qué nos indican sus multiplicidades:

- Para el campo *header*, el símbolo ! al final de la flecha, indica que cada objeto de la clase *List* está relacionado exactamente con un objeto de la clase *Entry* por el campo *header*. El símbolo ? al inicio de la flecha, indica que cada objeto *Entry* es el objeto *header* de un objeto *List* como máximo.

- Para el campo *element*, el símbolo ? al final de la flecha indica que el campo *element* de un objeto *Entry* apunta a cero o a uno de los objetos de la clase *Object*. Dicho de otro modo, éste puede ser *null*: un objeto *List* puede almacenar referencias *null*. La ausencia de un símbolo al inicio de la flecha, indica que un objeto puede estar apuntado por el campo *element* de cualquier número de objetos *Entry*. Es decir, una *List* puede almacenar duplicados.
- Para el campo *next*, el símbolo ! al final y al inicio de la flecha indica que el campo *next* de todo objeto *Entry* apunta a un objeto *Entry*, y todo objeto *Entry* queda apuntado por el campo *next* de un objeto *Entry*.

8.1.4 Mutabilidad

Hasta ahora, todas las características del modelo de objetos que hemos descrito restringen a los estados individuales. Las restricciones de mutabilidad describen cómo pueden alterarse los estados. Para mostrar que una restricción de multiplicidad se ha violado, es necesario exhibir un único estado, pero para exponer la violación de una restricción de mutabilidad, necesitamos mostrar dos estados que representen el estado anterior y posterior a la alteración global del estado. Las restricciones de mutabilidad se pueden aplicar a ambos conjuntos y relaciones, pero por ahora, tendremos en cuenta únicamente una forma limitada, en la cual una barra (figura de arriba) opcional se puede usar para marcar el final de una flecha de campo. Cuando está presente, esta marca indica que un objeto con el cual un determinado objeto está relacionado a través de un campo, debe ser siempre el mismo. En este caso, decimos que el campo es *immutable*, *estático*, o más exactamente, *target static* (o estático al final de la flecha, dado que más tarde facilitaremos un significado para una barra situada al inicio de la flecha).

En nuestro diagrama, por ejemplo, la barra al final de la relación *header* indica que un objeto *List*, una vez creado, siempre apunta a través de su campo *header* al mismo objeto *Entry*. Un objeto es inmutable si todos sus campos son inmutables. Se dice que una clase es inmutable si sus objetos son inmutables.

8.1.5 Diagramas de instancia

El significado de un modelo de objetos es una colección de configuraciones, es decir, todas las que satisfacen las restricciones del modelo. Éstas se pueden representar en *diagramas de instancia* o *snapshots* (un *snapshot* es una representación simplificada), que son simplemente grafos que se componen de objetos y referencias que los conectan. Cada objeto está etiquetado con la clase (la más específica) a la que pertenecen. Cada referencia está etiquetada con el campo que representa.

La relación entre un *snapshot* y un modelo de objeto es igual que la relación entre una instancia de un objeto y una clase, o como la relación entre una sentencia y la gramática.

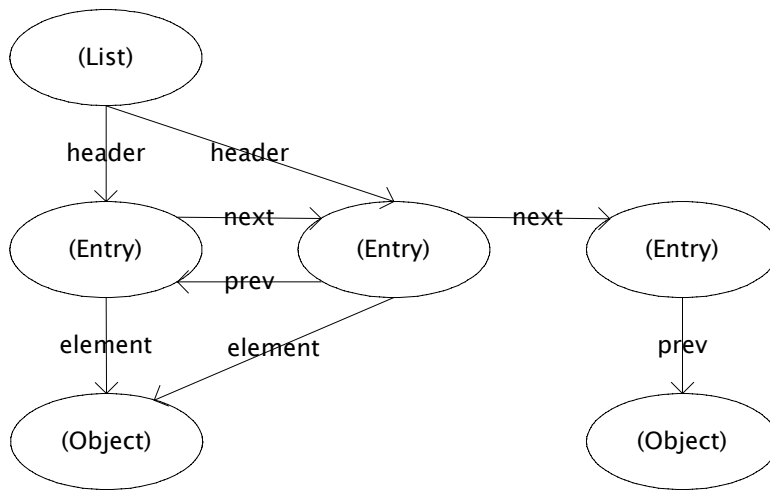
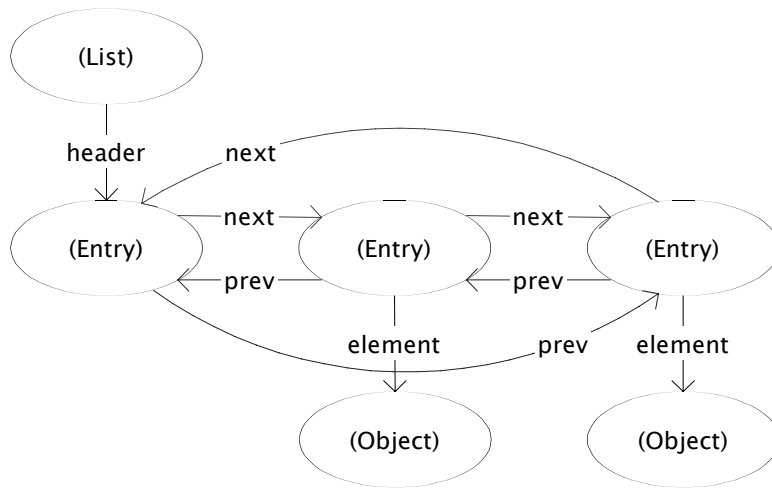
La figura de abajo muestra un *snapshot* legal (que pertenece a la colección representada por el modelo de objeto de los ejemplos de arriba) y uno ilegal (que no pertenece a la colección). Existe, por supuesto, un número infinito de *snapshots* legales, ya que se puede elaborar una lista de cualquier longitud.

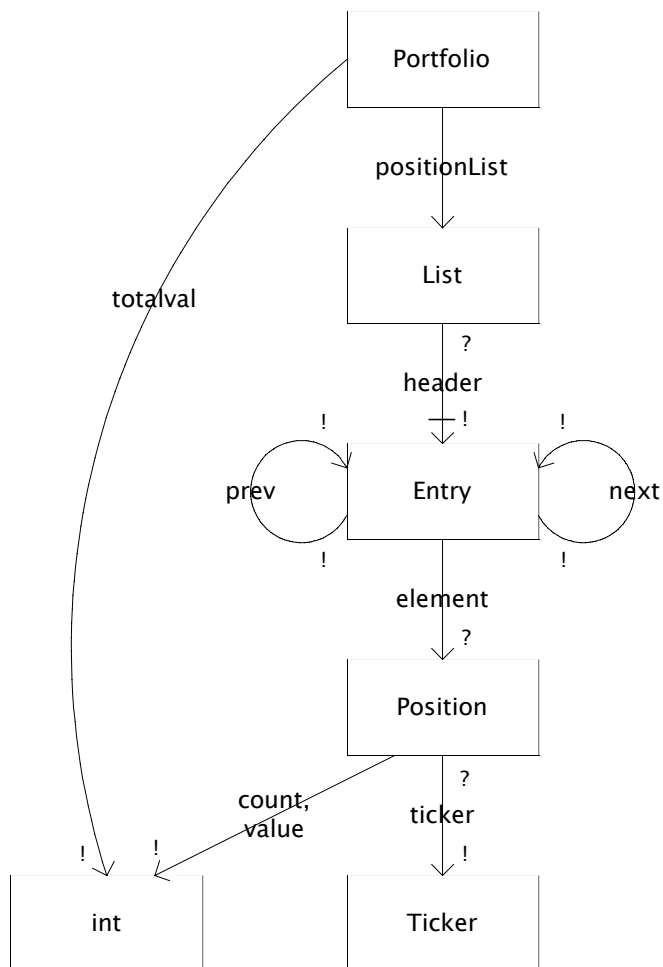
Un ejercicio práctico para comprobar que usted comprende el significado del modelo de objetos, es analizar el *snapshot* ilegal y definir qué restricciones viola. Las restricciones son las de multiplicidad y las que están implícitas en la colocación de las flechas. Por ejemplo, ya que la flecha del campo *header* va desde *List* hasta *Entry*, un *snapshot* que contenga un campo etiquetado con una flecha de referencia, partiendo de *Entry* hasta *Entry*, debe ser erróneo. Observe que las restricciones de mutabilidad no son pertinentes aquí; le indican las transiciones permitidas.

8.2 Modelos de programas completos

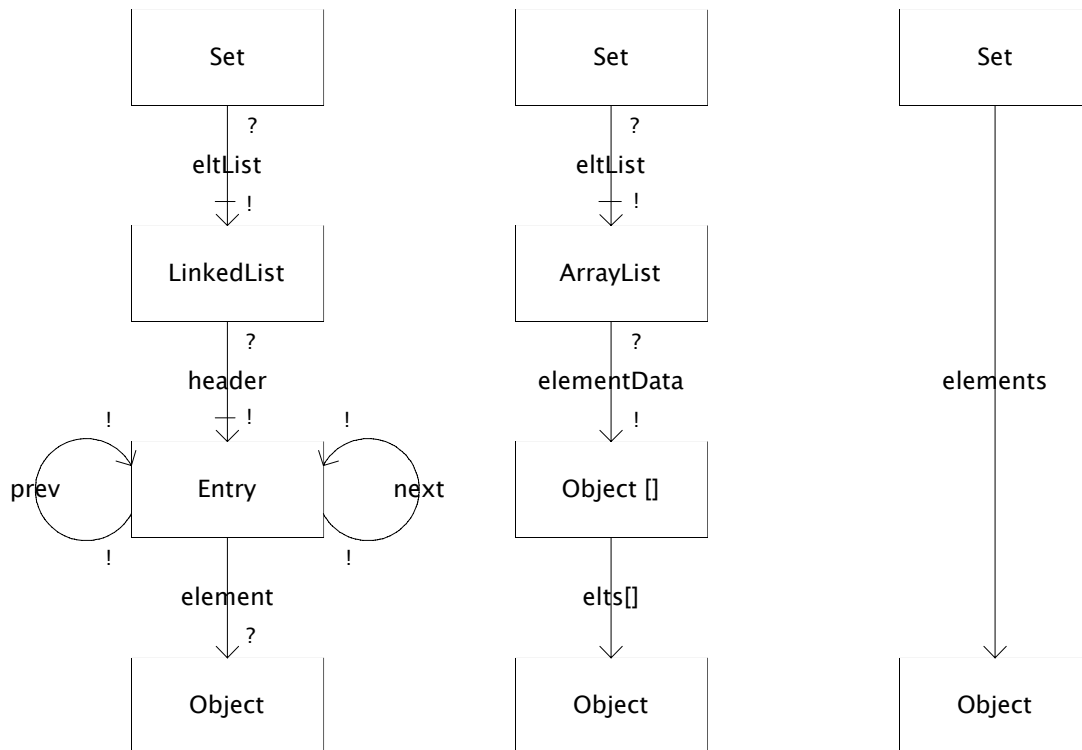
Un modelo de objeto puede utilizarse para mostrar cualquier parte del estado de un programa. En el ejemplo *List* de arriba, nuestro modelo de objeto exhibía únicamente los objetos implicados en la representación del tipo abstracto *List*. Sin embargo, en realidad, los modelos de objeto resultan más útiles cuando incluyen objetos de muchos tipos, ya que capturan la interrelación entre éstos, que constituye a menudo la esencia de un diseño orientado a objetos.

Suponga, por ejemplo, que estamos construyendo un programa para controlar los precios de las acciones de la bolsa. Podemos diseñar un tipo de datos llamado *Portfolio* que represente a una cartera de un determinado tipo de acciones de bolsa. Un *Portfolio* contiene una lista de objetos de tipo *Position*, cada uno de los cuales posee un símbolo *Ticker* para una determinada acción, el recuento del número de acciones de la cartera y el valor actual para cada acción. El objeto *Portfolio* también mantiene el valor total de todas las posiciones indicadas por los objetos *Positions*.





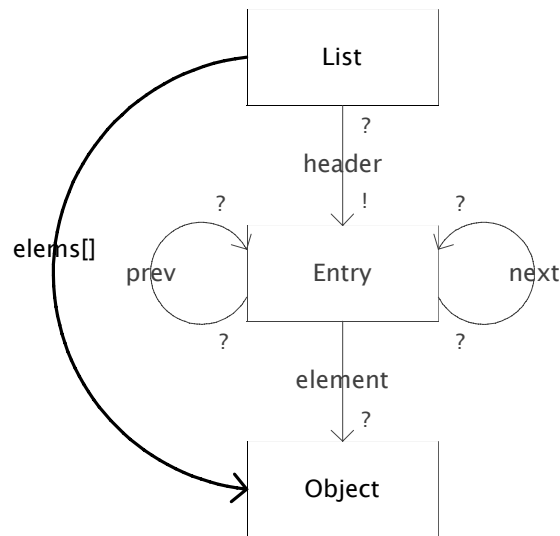
En el modelo de objeto de abajo se puede observar esto. Observe cómo los objetos *Entry* apuntan ahora a los objetos *Position*: pertenecen a una lista (objeto *List*) de objetos *Position*, que no es una lista cualquiera. Debemos permitir que haya varios recuadros en el mismo diagrama con la etiqueta *List*, que se correspondan con distintos tipos de *List*. Y consecuentemente, debemos ser un poco cuidadosos sobre cómo interpretamos las restricciones implícitas en una flecha correspondiente a un campo. La flecha marcada como *element*, que parte de *Entry* hacia *Position* en nuestro diagrama, por ejemplo, no significa que todo objeto *Entry* del programa, apunte a un objeto *Position*, sino que todo objeto *Entry* contenido en un objeto *List*, que está contenido a su vez en el objeto *Portfolio*, apunta a un objeto *Position*.



8.3 Puntos de vista concretos y abstractos

njunto de la forma de un tipo de dato abstracto. En algunas circunstancias, por ejemplo, cuando tenemos muchos conjuntos pequeños que representan un conjunto como una lista, es una opción aceptable. La figura anterior muestra tres modelos de objeto. Los dos primeros son dos versiones de un tipo llamado *Set*, uno representado con un *LinkedList* y otro con un *ArrayList*. (Pregunta para el lector astuto: ¿por qué es el campo *header* en la representación con *LinkedList* inmutable y, sin embargo, no sucede lo mismo con el campo *elementData*, en la representación con *ArrayList*?).

Si lo que nos interesa es saber cómo se representa *Set*, sería posible que quisiéramos mostrar estos modelos de objeto. Pero si nuestro interés se centra en el papel que *Set* representa dentro de un programa mayor y no queremos preocuparnos por la elección de la representación, preferiríamos un modelo de objeto que ocultase la diferencia entre estas dos versiones. El tercer modelo de objeto, a mano derecha, es este mismo modelo, que sustituye todos los detalles de la representación de *Set*, con un único campo denominado *elements*, que conecta objetos *Set* directamente con sus elementos.

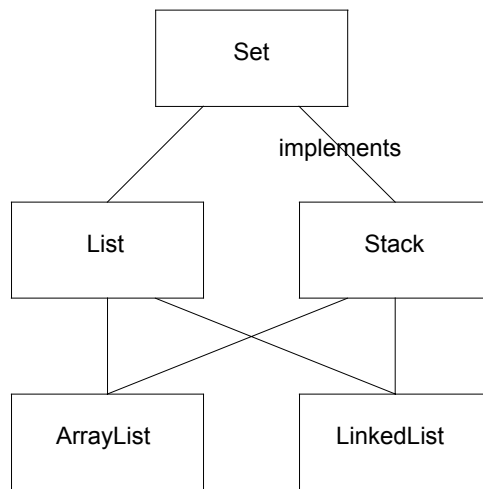


Este campo no corresponde a un campo declarado en Java, en la clase *Set*; se trata de un campo *abstracto* o de *especificación*.

Por tanto, se pueden diseñar muchos modelos de objetos para el mismo programa. Usted goza de libertad para decidir cuánto modelará de un estado y, para esa parte del estado, qué nivel de abstracción tendrá su representación. Sin embargo, existe un nivel específico de abstracción que está establecido como norma. Éste sería el nivel presentado por los métodos en el código. Por ejemplo, si algún método de la clase *Set* devuelve un objeto de tipo *LinkedList*, no tendría apenas sentido realizar una abstracción de la clase *LinkedList*. Pero si, desde el punto de vista de un cliente de *Set*, resulta imposible saber si se está utilizando un *LinkedList* o un *ArrayList*, sería más lógico mostrar el campo abstracto *elements*.

Un tipo abstracto puede estar representado por muchos tipos de representación distintos. Asimismo, un tipo se puede utilizar para representar muchos tipos abstractos diferentes. Por ejemplo, una lista encadenada se puede usar para implementar una pila: a diferencia de la interfaz genérica *List*, *LinkedList* ofrece los métodos *addLast* y *removeLast*. Además, por cuestiones de diseño, *LinkedList* implementa directamente la interfaz *List*, que representa una secuencia abstracta de elementos. Podemos por tanto, observar a la clase *LinkedList*, de manera más abstracta con un campo *elems[]*, escondiendo la estructura interna *Entry*, en la cual, el símbolo *[]* indica que el campo *elems* representa una secuencia indexada.

La figura de abajo muestra estas relaciones: una flecha indica “puede utilizarse para representar”. Obviamente, no estamos ante una relación simétrica. Generalmente, el tipo concreto posee más información en su contenido: una lista puede representar un conjunto, pero un conjunto no puede representar una lista. La razón es que un



conjunto no puede contener información relativa al orden, o permitir duplicados. Observe también que ningún tipo es inherentemente “abstracto” o “concreto”.

Estas nociones son relativas. Una lista es abstracta con respecto a una lista encadenada, utilizada para representarla, pero es concreta con respecto a un conjunto que ésta represente.

8.3.1 Funciones de abstracción

Debido a una elección específica de un tipo abstracto y concreto, podemos mostrar cómo los valores del tipo concreto se interpretan como valores abstractos mediante el uso de una función de abstracción, como se explicó en un tema anterior.

Recuerde que el mismo valor concreto puede interpretarse de modos distintos, por tanto, la función de abstracción no está determinada por la elección de tipos concretos y abstractos. Se trata de una decisión de diseño y determina cómo se escribe el código para métodos del tipo de dato abstracto.

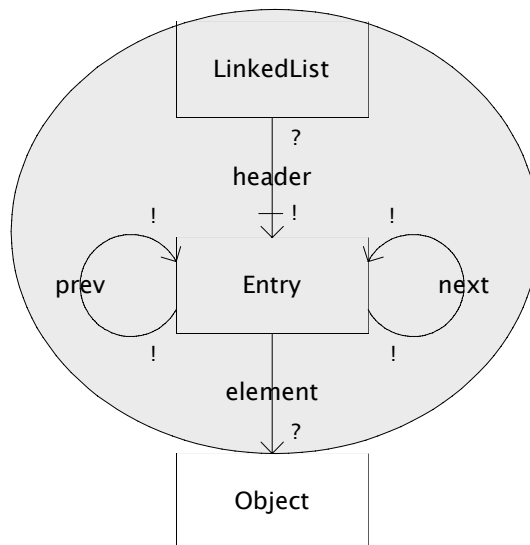
En un lenguaje de programación sin objetos mutables, en el que no tuviésemos que preocuparnos por el reparto, podríamos interpretar los “valores” abstractos y concretos como simplemente como eso: valores). La función de abstracción es claramente, por tanto, una función matemática. Piense, por ejemplo, en las varias formas a través de las cuales los enteros se representan como *bitstrings*. Cada una de estas representaciones pueden ser descritas como una función de abstracción desde *bitstring* hasta *integer*. Una codificación que coloque al menos significativo primero, por ejemplo, puede tener una función de asociación como:

$$A(0000) = 0$$

...

$$A(0001) = 8$$

$$A(1001) = 9$$



...

Sin embargo, en un programa orientado a objetos en el que tengamos que preocuparnos por cómo las alteraciones a un objeto a través de una ruta (un método, por ejemplo) pueden afectar a una visión del objeto a través de otra ruta, los “valores” son, de hecho, como subgrafos pequeños. El modo más claro de definir la función de abstracción en estas circunstancias consiste en facilitar una regla para cada campo abstracto, explicando cómo se obtiene a partir de campos concretos. Por ejemplo, para la representación *LinkedList* de *Set*, podemos escribir

s.elements = s.list.header.*next.element,

para expresar, que para cada objeto *s* de la clase, los objetos apuntados por el campo abstracto *elements*, son objetos obtenidos al seguir *list* (el objeto *List*), *header* (para el primer objeto *Entry*), luego cero o más transversales por el campo *next* (hasta los demás objetos *Entry*) y, para cada uno de estos, seguir el campo *element* una vez (hasta el objeto apuntado por *Entry*). Observe que esta regla es, por sí misma, una especie de modelo de objeto invariante: le indica donde está permitido colocar flechas etiquetadas con *elements* dentro de un *snapshot*.

En general, un tipo abstracto puede tener cualquier número de campos abstractos, y la función de abstracción se especifica al dar una regla para cada uno de estos campos.

En la práctica, a excepción de unos pocos tipos *container*, las funciones de abstracción, por lo general, son más problemáticas que útiles. Sin embargo, comprender la idea de función de abstracción es algo valioso, ya que le ayudará a asimilar el concepto de abstracción de datos.

Además, debería estar preparado para escribir una función de abstracción si surge la necesidad. La *fórmula booleana* en *CNF* del tema 6, es un buen ejemplo de un tipo abstracto que realmente necesita una función de abstracción. En ese caso, sin una firme comprensión de la función de abstracción, es complicado conseguir un código correcto.

8.3.2 Invariantes de representación

Un modelo de objeto es un tipo de invariante: una restricción válida durante toda la vida de un programa. Un invariante de representación o “invariante Rep”, como vimos en el tema 6, es un tipo específico de invariante que describe si la representación de un objeto abstracto está bien formada. Algunos aspectos de un invariante Rep pueden expresarse en un modelo de objeto. Sin embargo, existen otros que no se pueden expresar de forma gráfica. Además, no todas las restricciones de un modelo de objeto son invariantes rep.

Un invariante Rep es una restricción que puede aplicarse a un único objeto de un tipo abstracto, y le indica si la representación es correcta. Por tanto, un invariante siempre implica exactamente un objeto del tipo abstracto en cuestión, y cualquiera de los objetos que puedan ser alcanzados por su representación.

Podemos trazar un contorno alrededor de una parte del modelo de objeto para indicar que un invariante de representación en concreto se refiere a esta parte. Este contorno agrupa los objetos de una representación junto con su objeto abstracto. Por ejemplo, para el invariante Rep de *LinkedList* visto como un *List* (es decir, una secuencia abstracta de elementos), este contorno incluye los elementos *Entry*. Como era de esperar, las clases dentro del contorno, son las clases abstraídas por el campo *elems[]*. Del mismo modo, el invariante Rep para la clase *ArrayList* engloba al *Array* contenido.

Los detalles de los invariantes rep se trataron en el tema 6: para *LinkedList*, por ejemplo, los invariantes incluyen restricciones como la necesidad de los objetos *Entry* de formar un ciclo, o de que *header* esté siempre presente y que tenga un campo *element* con valor *null*, etc.

Recordemos por qué el invariante Rep es útil, por qué no es solamente un concepto teórico, sino una herramienta práctica:

- El invariante de representación captura, en un determinado lugar, las reglas sobre cómo se forma un valor legal de la representación. Si usted está modificando el código de un *ADT* (tipo de datos abstracto) o escribiendo un método nuevo, es necesario que sepa qué invariantes deben restablecerse y en cuáles puede confiar. El invariante Rep le indica todo lo que necesita saber; esto es lo que se persigue en el razonamiento *modular*. Si no hay un registro explícito de un invariante Rep, tendrá que leer el código de cada método.
- El invariante Rep captura la esencia del diseño de la representación. La presencia de la entidad *header* y la forma cíclica de *LinkedList* de Java, por ejemplo, son buenas decisiones de diseño que hacen que los métodos sean más fáciles de codificar de modo uniforme.

- Como veremos en los próximos temas, el invariante Rep se puede utilizar para detectar errores en tiempo de ejecución en una especie de “programación defensiva”.

8.3.3 Exposición de representación

El invariante Rep proporciona un razonamiento modular mientras que la representación sea modificada únicamente dentro de la clase del tipo de dato abstracto. Si existe la posibilidad de modificaciones a través de un código externo a la clase, hace falta examinar el programa entero para asegurarnos de que el invariante Rep se está manteniendo.

Esta desagradable situación se conoce como *exposición de representación*.

Hemos visto en temas anteriores algunos ejemplos claros y más sutiles. Un ejemplo sencillo se da cuando un tipo de dato abstracto proporciona acceso directo a uno de los objetos que está dentro del contorno del invariante Rep. Por ejemplo, cada implementación de la interfaz de *List* (en realidad, la interfaz *Collection* más general) debe proporcionar un método que devuelva la lista como un *array* de elementos.

```
public Object [] toArray ()
```

La especificación de este método dice:

El array devuelto estará “seguro”, en cuanto a que este objeto Collection no mantendrá ninguna referencia al array. (O que este método debe asignar un nuevo array incluso si este objeto Collection está respaldado por un array). El llamador tiene por tanto, libertad para modificar el array devuelto.

En la implementación de *ArrayList*, el método se implementa como:

```
private Object elementData[];
...
public Object[] toArray() {
    Object[] result = new Object[size];
    System.arraycopy(elementData, 0, result, 0, size);
    return result;
}
```

Observe cómo el *array* interno se ha copiado para que se produzca el resultado. Si, por el contrario, el *array* se hubiese devuelto inmediatamente, como en este caso,

```
public Object[] toArray() {
    return elementData;
}
```

hubiésemos obtenido una exposición de representación. Las modificaciones posteriores al *array* desde fuera del tipo abstracto afectarían a la representación interna. (De hecho, en este caso, tenemos un invariante Rep tan débil, que un cambio al *array* no podría romperlo, y esto produciría un efecto tan extraño como ver que el valor de la lista abstracta cambia mientras se modifica el *array*).

Sin embargo, podríamos imaginar una versión de *ArrayList* que no almacenase referencias nulas; en este caso, la asignación del valor *null* a un elemento del *array* arruinaría el invariante).

Ahora presentamos un ejemplo mucho más sutil. Suponga que implementamos un tipo de dato abstracto para listas sin duplicados y que definimos el concepto de duplicación a través del método *equals* de los elementos. Ahora, nuestro invariante Rep indicará para la representación de una lista encadenada, por ejemplo, que no hay ningún par de objetos *Entry* distintos cuya prueba de igualdad devuelva el valor *true*. Si los elementos son mutables y el método *equals* examina los campos internos, es posible que la alteración de un elemento, haga que el elemento alterado sea igual que otro. Por tanto, el acceso a los elementos propiamente dichos constituirá una exposición de representación.

Esto, en realidad, no es distinto al caso sencillo, dado que el problema consiste en acceder a un objeto dentro del contorno. El invariante en este caso, ya que depende del estado interno de los elementos, posee un contorno que incluye los elementos de tipo *Object*. La igualdad crea cuestiones especialmente complicadas; nos dedicaremos a ello en el tema de mañana.